

Time Complexity

Mastery

Q1 – Q30 Detailed Question Bank & Solutions

Iterative Analysis | Recursion | Recurrence Relations

| Iterative Analysis: Linear & Quadratic



Q1: Simple Loop

```
for(i=0; i<n; i++)  
    // ...  
Loop runs exactly n times.
```

$\Theta(n)$



Q2: Nested Square

```
for(i=0; i<n; i++)  
    for(j=0; j<n; j++)  
        // ...  
Outer n × Inner n.
```

$\Theta(n^2)$



Q3: Dependent Nested

```
for(i=0; i<n; i++)  
    for(j=0; j<i; j++)  
        // ...  
Sum: 0+1+2+...+(n-1)
```

$\Theta(n^2)$

| Logarithmic Scaling

Q#	Code Snippet	Explanation	Complexity
Q4	<code>for(i=1; i<=n; i*=2)</code>	Doubling i reduces iterations to log steps.	$\Theta(\log n)$
Q5	<code>for(i=n; i>1; i/=2)</code>	Halving i until it reaches 1.	$\Theta(\log n)$
Q6	<code>for(i=1; i<=n; i++) for(j=1; j<=log(n); j++)</code>	Outer n runs inner $\log n$ times.	$\Theta(n \log n)$

| While Loops & Separated Work



Q7: Decrementing While

```
while(n > 0) { n--; }
```

N steps total.

$\Theta(n)$

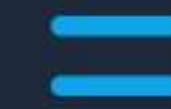


Q8: Halving While

```
while(n > 1) { n /= 2; }
```

Repeated division by 2.

$\Theta(\log n)$



Q9: Separate Loops

```
for(i=1; i≤n; i++)  
for(j=1; j≤n; j++)  
n + n = 2n → Drop constants.
```

$\Theta(n)$

| Dominance & Polynomial Growth

Q#	Logic	Dominant Term / Math	Complexity
Q10	<code>Loop(n) + Loop(n²)</code>	$n + n^2 \rightarrow n^2$ dominates.	$\Theta(n^2)$
Q11	<code>Nested(n × n²)</code>	$n \times n^2$ results in cubic growth.	$\Theta(n^3)$
Q12	<code>for(i=1; i≤n; i*=3)</code>	log base 3 of n.	$\Theta(\log n)$
Q13	<code>Nested(i² inside i)</code>	Sum of squares: $\sum i^2 = [n(n+1)(2n+1)]/6$	$\Theta(n^3)$

Part II: Recursion

Analyzing the call stack and branching factors.

Recursive Reductions

Q14: Linear Reduction

`fun(n-1)`

Recurrence: $T(n) = T(n-1) + 1$

$\Theta(n)$

Q15: Log Reduction

`fun(n/2)`

Recurrence: $T(n) = T(n/2) + 1$

$\Theta(\log n)$

Q16 & Q17: Binary Branching

`fun(n-1)` twice or `fib(n-1)+fib(n-2)`

Exponential growth tree.

$\Theta(2^n)$

| Stack Depth & Output Complexity

Q#	Function Pattern	Stack Depth / Path	Complexity
Q18	<code>n * fact(n-1)</code>	n calls, constant work per call.	$\Theta(n)$
Q19	<code>fun(n-1); cout<</code>	Single recursive path of length n.	$\Theta(n)$
Q20	<code>fun(n/2); cout<</code>	Halving path. log n calls.	$\Theta(\log n)$

| Solving Recurrence Relations

Q21

$$T(n) = T(n-1) + 1$$

$$T(n) = \Theta(n)$$

Q22

$$T(n) = T(n-1) + n$$

$$T(n) = 1 + 2 + \dots + n = \Theta(n^2)$$

Q23

$$T(n) = T(n/2) + 1$$

$$T(n) = \Theta(\log n)$$

| Master Theorem Analysis

Q24: Exponential

$$T(n) = 2T(n-1) + 1$$

Repeated doubling leads to exponential scale.

$$\Theta(2^n)$$

Q25: Master Theorem

$$T(n) = T(n/2) + n$$

$$a=1, b=2, f(n)=n$$

$$f(n) \text{ dominates } n^{\log_2 1} = 1$$

$$\Theta(n)$$

| Advanced / GATE-Level Challenges

Q#	Loop Context	Work Logic	Complexity
Q26	i: 1 to n j: 1 to i ($j^*=2$)	Sum of $\log(i)$ from 1 to n	$\Theta(n \log n)$
Q27	i: $\log n$ steps j: n steps	$\log n \times n$	$\Theta(n \log n)$
Q28	i: n to 1 j: i to 1	Triangular Sum $n(n+1)/2$	$\Theta(n^2)$
Q29	i: 1 to n j: 1 to n ($j^*=2$)	$n \times \log n$	$\Theta(n \log n)$

| Q30: Recursive Tree Expansion

Q30 Case Study

```
void fun(n) { if(n≤1) return; fun(n/2); fun(n/2); }
```

Recurrence: $T(n) = 2T(n/2) + 1$

Using Master Theorem: $a=2, b=2, n^{\log_2 2} = n$

Case 1: n dominates $f(n)=1$.

$\Theta(n)$

Problem Set Complete.